Longest Increasing Subsequence

AlgoExpert · Very Hard · Dynamic Programming

The problem

- One strictly increasing subsequence of maximum length (patience sorting with predecessor links).

Intuition

- Patience sorting: keep the smallest possible tail for each length and binary search where each number goes.

Approach

- 1. Tails array of indices
- 2. Binary search the position
- 3. Record predecessors
- 4. Rebuild from the last tail

Java solution

```java
public class LongestIncreasingSubsequence {
    public static List<Integer> longestIncreasingSubsequence(int[] a) {
        int n = a.length, len = 0;
        int[] tails = new int[n], prev = new int[n];
        for (int i = 0; i < n; i++) {
            int lo = 0, hi = len;
            while (lo < hi) { int m = (lo + hi) >>> 1; if (a[tails[m]] < a[i]) lo = m + 1; else hi = m; }
            prev[i] = lo > 0 ? tails[lo - 1] : -1;
            tails[lo] = i;
            if (lo == len) len++;
        }
        List<Integer> out = new ArrayList<>();
        for (int i = len == 0 ? -1 : tails[len - 1]; i != -1; i = prev[i]) out.add(a[i]);
        Collections.reverse(out);
        return out;
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(n)

Recap

- Patience sorting: keep the smallest possible tail for each length and binary search where each number goes.
- Complexity: O(n log n)